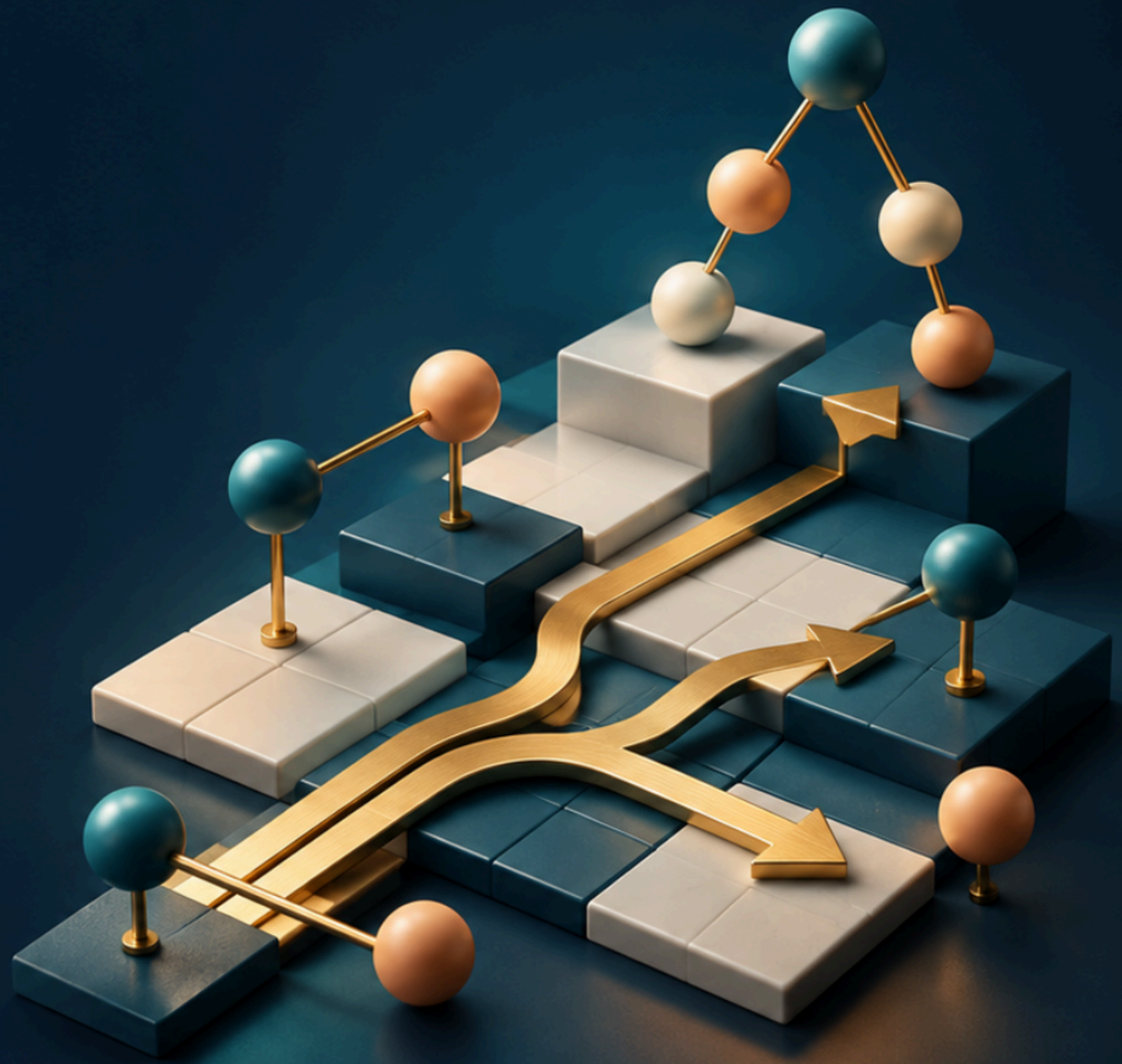
10

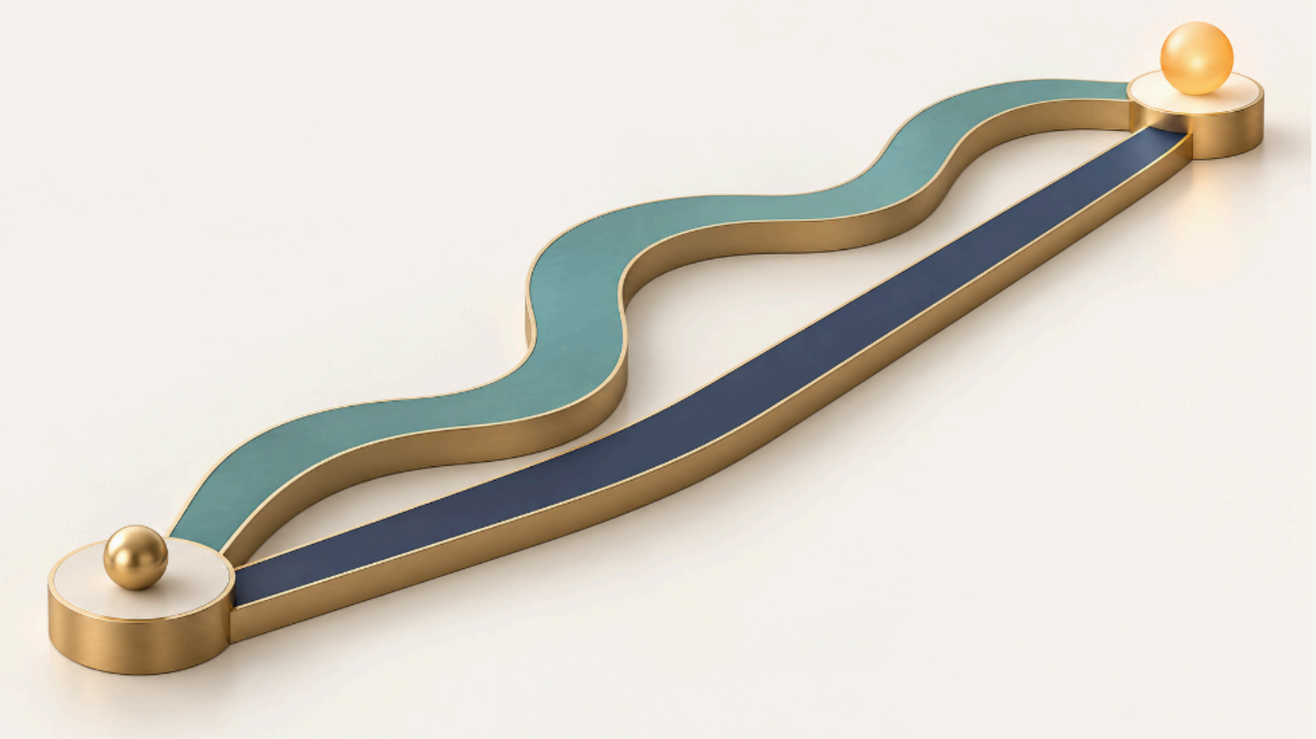
근사 알고리즘

최적 답을 구하기 어려울 때, 빨리 답을 구하면서 얼마나 나쁠 수 있는지도 봅니다.



근사 알고리즘

최적 답을 구하기 어려울 때, 빨리 답을 구하면서 얼마나 나쁠 수 있는지도 봅니다.



최적 비용이 10이라면 두 배 기준은 20이다.

빨리 구한 답의 비용이 기준 안에 드는지 확인한다. 그림은 같은 목표에 이르는 서로 다른 선택을 나타낸다.

10-1 최적 비용의 허용 배수 확인

비용이 작을수록 좋은 문제에서, 이미 구한 해의 비용 cost와 최적 비용 opt가 주어진다. cost가 opt의 factor배 이내인지 검사한다. 최적 비용을 새로 구할 필요는 없다.

01

cost≤opt×factor이면 1, 아니면 0을 반환하시오.

F10-01 cost, opt, factor

10-1 입력과 반환값

Python: within_factor(cost, opt, factor)

cost: 구한 해의 비용. 1~1,000,000의 정수
opt: 알려진 최적 비용. 1≤opt≤cost
factor: 허용하는 배수. 1~100의 정수

반환값

허용 범위 안이면 정수 1, 범위를 넘으면 정수 0이다.

선택·판정 조건

허용 비용과 정확히 같을 때도 1이다.
factor=1이면 최적 비용과 같아야 통과한다.

10-1 두 배보다 작음 · 경계와 같음

두 배보다 작음

```
cost=18, opt=10, factor=2
```

반환값 1

허용 비용은 20이고 18은 그 이하이다.

경계와 같음

```
cost=20, opt=10, factor=2
```

반환값 1

20≤20이므로 통과한다.

10-1 경계를 넘음 · 최적 비용과 같음

경계를 넘음

```
cost=21, opt=10, factor=2
```

반환값 0

21은 허용 비용 20을 초과한다.

최적 비용과 같음

```
cost=10, opt=10, factor=1
```

반환값 1

최적 비용 자체이므로 통과한다.

10-1 1배 조건에서 더 비쌈

1배 조건에서 더 비쌈

`cost=11, opt=10, factor=1`

반환값

0

허용 비용 10보다 크다.

10-1 예제의 계산 과정

구한 비용 cost	허용 비용 opt×factor	비교	답
18	20	18≤20	1
20	20	20≤20	1
21	20	21>20	0

정확히 두 배인 20도 허용합니다.

10-1 나누지 않고 허용 비용과 비교한다

최적 비용이 10이고 두 배를 허용하면 허용 비용은 20이다. cost를 20과 비교하면 된다. 정수 나눗셈으로 cost/opt를 먼저 구하면 소수 부분을 버려 경계를 잘못 판단할 수 있다. 곱셈과 이하 비교로 직접 계산한다.

10-1 코드에서 사용할 값

변수	예제의 값
cost	구한 해의 비용 18
opt	가능한 가장 싼 비용 10
factor	허용 배수 2
cost <= opt*factor	18≤20이므로 참

근사 알고리즘에서는 빨리 구한 답의 비용이 최적 비용보다 얼마나 큰지도 봅니다.

10-1 Python

```python
def within_factor(cost, opt, factor):
    return int(cost <= factor*opt)
```

시간 O(1), 추가 공간 O(1)

10-1 C 함수의 인자

함수 원형

int within_factor(int cost, int opt, int factor)

배열과 작업 공간

세 인자는 Python과 같은 뜻이다.

10-1 C

```
1  int within_factor(int cost, int opt, int factor) {
2      return cost<=factor*opt;
3  }
```

시간 O(1), 추가 공간 O(1)

10-2 가장 한가한 기계에 작업 맡기기

같은 성능의 기계 m대가 시간 0부터 준비되어 있다. 작업 시간 배열 jobs를 앞에서부터 읽으며, 누적 작업 시간이 가장 작은 기계에 그 작업을 맡긴다. 누적 시간이 같으면 번호가 작은 기계를 고른다.

02

아래 배정 규칙대로 모든 작업을 맡긴 뒤, 전체 작업이 끝나는 시각을 반환하시오.

F10-02 jobs, m

10-2 입력과 반환값

Python: list_scheduling(jobs, m)

jobs: 길이 0~1000의 작업 시간 배열. 각 시간은 0~1,000,000
m: 기계 수. 1~10의 정수. 번호는 0~m-1

반환값

기계별로 맡긴 시간의 합 중 최댓값을 반환한다. 기계들의 완료 시각을 모두 더하지 않는다.

선택·판정 조건

작업의 입력 순서를 바꾸지 않는다. 한 작업을 여러 기계로 나눌 수 없다.
각 기계는 자신에게 배정된 작업을 받은 순서대로 계속 처리한다.
작업이 없으면 완료 시각은 0이다.

10-2 3,2,4 순서 · 4,3,2 순서

3,2,4 순서

```
jobs=[3,2,4], m=2
```

반환값 6

기계별 합은 [3,6]이다. 둘 다 끝나는 시각은 6이다.

4,3,2 순서

```
jobs=[4,3,2], m=2
```

반환값 5

기계별 합은 [4,5]이다. 같은 작업들이어도 입력 순서가 달라 답은 5이다.

10-2 기계 한 대 · 동점 발생

기계 한 대

`jobs=[3,2,4], m=1`

반환값 9

한 기계가 세 작업을 모두 처리한다.

동점 발생

`jobs=[2,2,2], m=2`

반환값 4

마지막 작업 직전 합이 [2,2]라 번호가 작은 기계0에 맡긴다.

10-2 작업 없음

작업 없음

```
jobs=[], m=3
```

반환값

어느 기계에도 작업을 맡기지 않는다.



10-2 예제의 계산 과정

이번 작업 길이	작업 전 기계별 합	맡길 기계	작업 후 합
3	[0,0]	0	[3,0]
2	[3,0]	1	[3,2]
4	[3,2]	1	[3,6]

마지막 작업 직전에는 기계1의 합 2가 기계0의 합 3보다 작습니다.

10-2 기계별 합 [3,2]에서 4를 어디에 맡길까

첫 작업 3은 기계0, 다음 작업 2는 기계1에 맡긴다. 이때 기계별 누적 시간은 [3,2]이다. 마지막 작업 4는 합이 더 작은 기계1에 맡겨 [3,6]이 된다. 기계0은 시각 3에, 기계1은 시각 6에 끝나므로 전체 완료 시각은 6이다.

10-2 코드에서 사용할 값

변수	뜻
load[i]	기계 i에 지금까지 맡긴 시간의 합
machine	이번 작업을 맡길 기계 번호
load[machine] += time	선택한 기계의 합에 이번 작업 시간 추가
max(load)	가장 늦게 끝나는 기계의 완료 시각

누적 시간이 같으면 번호가 작은 기계를 먼저 고릅니다.

10-2 Python

```python
def list_scheduling(jobs, m):
    load = [0]*m
    for time in jobs:
        machine = 0
        for i in range(1, m):
            if load[i] < load[machine]:
                machine = i
        load[machine] += time
    return max(load)
```

작업 n개, 기계 m대일 때 시간 O(nm), 공간 O(m)

10-2 C 함수의 인자

함수 원형

int list_scheduling(const int jobs[], int n, int m)

배열과 작업 공간

jobs는 작업 시간 배열, n은 작업 수, m은 기계 수이다.

10-2 C

```c
int list_scheduling(const int jobs[], int n, int m) {
    int load[10]={0};
    for (int i=0; i<n; ++i) {
        int machine=0;
        for (int j=1; j<m; ++j)
            if (load[j]<load[machine]) machine=j;
        load[machine]+=jobs[i];
    }
    int answer=0;
    for (int j=0; j<m; ++j)
        if (load[j]>answer) answer=load[j];
    return answer;
}
```

작업 n개, 기계 m대일 때 시간 O(nm), 공간 O(m)

10-3 선의 양 끝을 고르는 정점 커버

무방향 그래프의 선들을 주어진 순서대로 확인한다. 이번 선의 양 끝 점을 모두 아직 고르지 않았다면 두 점을 함께 고른다. 한쪽이라도 이미 골랐다면 그 선에서는 아무 점도 추가하지 않는다.

03

이 절차가 끝난 뒤 선택한 점의 개수를 반환하시오.

`F10-03` `n, edges`

10-3 입력과 반환값

Python: cover_size(n, edges)

n: 점 수. 1~100의 정수. 번호는 0~n-1
edges: 0~1000개의 [끝점1,끝점2] 쌍. 두 끝점은 서로 다름

반환값

위 절차로 선택한 점의 수를 반환한다. 최솟값을 찾기 위해 절차를 바꾸지 않는다.

선택·판정 조건

선의 입력 순서를 유지한다. 순서가 바뀌면 고른 점의 수도 달라질 수 있다.
같은 선이 목록에 다시 나와도 이미 고른 점은 한 번만 센다.
선과 연결되지 않은 점은 선택할 필요가 없다. 선이 없으면 답은 0이다.

10-3 삼각형 · 양 끝 선을 먼저 봄

삼각형

`n=3, edges=[[0,1],[1,2],[0,2]]`

반환값 2

첫 선에서 0과 1을 고른 뒤 나머지 선은 모두 넘어간다.

양 끝 선을 먼저 봄

`n=4, edges=[[0,1],[1,2],[2,3]]`

반환값 4

0과 1을 고르고, 마지막 선에서 2와 3도 고른다.

10-3 가운데 선을 먼저 봄 · 같은 선 반복

가운데 선을 먼저 봄

`n=4, edges=[[1,2],[0,1],[2,3]]`

반환값 2

1과 2를 먼저 고르면 남은 두 선에서 추가할 점이 없다.

같은 선 반복

`n=2, edges=[[0,1],[0,1]]`

반환값 2

두 번째 선에서는 이미 고른 점들을 다시 세지 않는다.

10-3 선 없음

선 없음

`n=4, edges=[]`

반환값

점이 네 개 있어도 고른 점은 없다.

0

10-3 예제의 계산 과정

이번 선	현재 고른 점	처리	처리 후
0–1	없음	두 끝점을 모두 고름	0,1
1–2	0,1	1을 이미 골랐으므로 넘어감	0,1
0–2	0,1	0을 이미 골랐으므로 넘어감	0,1

9-2의 검사와 달리, 이번에는 규칙에 따라 점을 직접 고릅니다.

10-3 선 하나에서 고른 두 점이 다른 선도 덮는다

삼각형의 첫 선 0–1에서 두 끝을 고르면 0–2와 1–2도 선택한 점과 닿는다. 그래서 뒤의 두 선에서는 추가하지 않는다. 이 절차는 실제로 두 끝을 추가한 선들끼리 끝점이 겹치지 않게 만든다. 그런 선이 k개이면 어떤 커버도 최소 k점이 필요하고, 이 절차는 2k점을 골라 두 배 이내를 보장한다.

왜 두 배 이내인가요?

이런 선이 두 개라면

어떤 정점 커버도 각 선에 하나씩, 적어도 두 점을 골라야 합니다.

이 절차의 선택

각 선의 두 끝을 골라 총 네 점을 고릅니다.

같은 계산을 일반화

선이 k개이면 최소 k개가 필요하고, 우리는 2k개를 고릅니다.

10-3 코드에서 사용할 값

코드	뜻
u, v	이번 선의 양 끝 점 번호
chosen[u]	점 u를 골랐으면 True
not chosen[u] and not chosen[v]	두 점 모두 아직 고르지 않았음
answer += 2	새 점 두 개를 골랐으므로 개수에 2를 더함

이 절차가 만든 점의 수는 최소 정점 커버 크기의 두 배 이내입니다.

10-3 Python

```python
def cover_size(n, edges):
    chosen = [False]*n
    answer = 0
    for u, v in edges:
        if not chosen[u] and not chosen[v]:
            chosen[u] = True
            chosen[v] = True
            answer += 2
    return answer
```

간선 m개일 때 시간 O(n+m), 공간 O(n)

10-3 C 함수의 인자

함수 원형

int cover_size(int n, const int edges[][2], int m)

배열과 작업 공간

n은 정점 수, m은 간선 수이다. edges의 한 행이 두 끝점 번호이다.

10-3 C

```c
int cover_size(int n, const int edges[][2], int m) {
    int chosen[100]={0}, answer=0;
    for (int i=0; i<m; ++i) {
        int u=edges[i][0], v=edges[i][1];
        if (!chosen[u] && !chosen[v]) {
            chosen[u]=chosen[v]=1;
            answer+=2;
        }
    }
    return answer;
}
```

간선 m개일 때 시간 O(n+m), 공간 O(n)

10-4 새 원소를 가장 많이 덮는 집합 선택

전체 원소는 0부터 n−1까지이다. 여러 집합 중 아직 덮지 못한 원소를 가장 많이 포함하는 집합을 하나 고른다. 새로 덮는 수가 같으면 입력에서 먼저 나온 집합을 고른다. 고른 집합의 원소를 덮음으로 표시하고 이 과정을 반복한다.

04

모든 원소를 덮으면 선택한 집합 수, 끝까지 덮을 수 없으면 -1을 반환하시오.

`F10-04` `n, sets`

10-4 입력과 반환값

Python: greedy_set_cover(n, sets)

n: 전체 원소 수. 1~20의 정수
sets: 0~30개 집합을 나타내는 정수 배열. 각 값은 0~2^n–1

반환값

정해진 선택 절차의 횟수를 반환한다. 가능한 모든 조합 중 최소 집합 수를 구하는 문제는 아니다.

선택·판정 조건

집합은 원소 k가 들어 있으면 2^k를 더한 정수로 나타낸다. {0,1}은 1+2=3이다.
n=4일 때 3은 {0,1}, 6은 {1,2}, 12는 {2,3}을 나타낸다. 0은 빈 집합이다.
이미 덮은 원소는 새로 덮는 수에 포함하지 않는다.
남은 원소가 있는데 어느 집합도 새 원소를 덮지 못하면 –1이다.

10-4 두 집합으로 완료 · 동률에서 3을 먼저 고름

두 집합으로 완료

```
n=4, sets=[3,6,12]
```

반환값 2

처음에는 동률이라 3={0,1}을 고른다. 남은 {2,3}은 12로 덮는다.

동률에서 3을 먼저 고름

```
n=4, sets=[3,5,10]
```

반환값 3

3={0,1} 다음에는 5={0,2}와 10={1,3}이 각각 하나씩 더 필요하다.

10-4 같은 집합들의 순서 변경 · 덮을 수 없는 원소

같은 집합들의 순서 변경

`n=4, sets=[5,10,3]`

반환값 2

5={0,2}를 먼저 고르면 10={1,3}으로 바로 끝난다.

덮을 수 없는 원소

`n=4, sets=[3]`

반환값 -1

원소 2와 3은 어느 집합에도 없다.

10-4 집합이 없음 · 빈 집합은 도움 없음

집합이 없음

`n=4, sets=[]`

반환값 -1

전체 원소는 있는데 사용할 집합이 없다.

빈 집합은 도움 없음

`n=1, sets=[0,1]`

반환값 1

0은 아무것도 덮지 못한다. {0}을 뜻하는 1을 한 번 고른다.

10-4 예제의 계산 과정

남은 원소	A가 새로 덮음	B가 새로 덮음	C가 새로 덮음	선택
0,1,2,3	2	2	2	먼저 나온 A
2,3	0	1	2	C
없음	0	0	0	2회로 완료

동률이면 입력에서 앞에 있는 집합을 고릅니다.

10-4 집합 전체 크기 대신 새로 덮는 수를 센다

A={0,1}, B={1,2}, C={2,3}이면 처음에는 세 집합 모두 두 원소를 덮는다. A를 고른 뒤 남은 원소는 2와 3이다. 이때 B는 원소2 한 개, C는 원소2와3 두 개를 새로 덮으므로 C를 고른다. 비트마스크는 이 포함 여부를 정수의 0/1 자리로 적은 표현이다.

집합을 정수 3, 6, 12로 적는 이유

오른쪽부터 원소 0,1,2,3의 포함 여부를 적습니다.

집합	원소3	원소2	원소1	원소0	정수
A={0,1}	0	0	1	1	3
B={1,2}	0	1	1	0	6
C={2,3}	1	1	0	0	12
전체	1	1	1	1	15

가중치는 오른쪽부터 1,2,4,8입니다. A는 1+2=3입니다.

한 번 고른 뒤 남은 원소의 변화

전체 1111에서 A의 0011을 제거합니다.

계산	비트 표시	뜻
처음 remaining	1111	0,1,2,3이 남음
선택한 A	0011	0,1을 덮음
제거 후 remaining	1100	2,3만 남음
남은 것과 B의 공통 부분	1100 & 0110 = 0100	B는 2 하나만 새로 덮음

&는 두 표시가 모두 1인 자리만 남기는 연산입니다.

10-4 코드에서 사용할 값

변수	뜻
remaining	아직 덮지 못한 원소들의 표시
remaining & mask	현재 집합이 새로 덮을 수 있는 원소들
bit_count()	1로 표시된 원소의 수
remaining &= ~best	고른 집합의 원소들을 남은 목록에서 제거

비트마스크는 원소가 들어 있는지 0과 1로 적은 정수입니다.

10-4 Python

```python
def greedy_set_cover(n, sets):
    remaining = (1 << n)-1
    count = 0
    while remaining:
        best, gain = 0, 0
        for mask in sets:
            added = (mask & remaining).bit_count()
            if added > gain:
                best, gain = mask, added
        if gain == 0: return -1
        remaining &= ~best
        count += 1
    return count
```

단순 비트 개수 계산 기준 시간 $O(mn^2)$, 공간 $O(1)$

10-4 C 함수의 인자

함수 원형

int greedy_set_cover(int n, const int sets[], int m)

배열과 작업 공간

n은 전체 원소 수, m은 집합 수이다. sets의 길이는 m이다.

10-4 C (1/3)

```c
static int count_bits(int x) {
    int n=0;
    while (x) {
        n+=x&1;
        x>>=1;
    }
    return n;
}
```

10-4 C (2/3)

```c
int greedy_set_cover(int n, const int sets[], int m) {
    int remaining=(1<<n)-1, count=0;
    while (remaining) {
        int best=0, gain=0;
        for (int i=0; i<m; ++i) {
            int added=count_bits(sets[i]&remaining);
            if (added>gain) {
                best=sets[i];
                gain=added;
            }
        }
        if (!gain) return -1;
        remaining&=~best;
            ++count;
    }
```

10-4 C (3/3)

```
24          return count;
25      }
```

단순 비트 개수 계산 기준 시간 $O(mn^2)$, 공간 $O(1)$